Institut für Informationssicherheit und Verlässlichkeit
Prof. Dr. André Platzer platzer@kit.edu
Prof. Dr. Ralf H. Reussner reussner@kit.edu
Prof. Dr. Ina Schaefer ina.schaefer@kit.edu

Klausur Programmierparadigmen

SS24, 27. September 2024, 11:00 – 13:00 Uhr

Zugelassene Hilfsmittel:
Papierbasierte Quellen (Vorlesungsfolien, Übungsblätter, eigene Aufzeichnungen, Bücher, ...)
Die Verwendung von elektronischen Geräten ist verboten.

Bearbeitungszeit: 120 min

Aufgabe	max. Punkte	err. Punkte
1	10	
2	24	
3	18	
4	10	
5	15	
6	10	
7	15	
8	18	
Σ	**120**	
$\lceil\Sigma\rceil$	**120**	

Jeder Punkt entspricht ca. 1 min Bearbeitungszeit. Es ist garantiert, dass die Klausur mit 60 Punkten bestanden ist, und dass 115 Punkte für die Note 1,0 ausreichen.

Name: _____

Matrikelnummer: _____

Studiengang: _____

Schreiben Sie Ihre Lösungen direkt in die Klausur. Beschriften Sie **alle verwendeten Blätter** mit Ihrem Namen und Ihrer Matrikelnummer. Trennen Sie die geklammerten Blätter **nicht** auf. Weitere Blätter erhalten Sie bei Bedarf. Wir behalten uns vor, Lösungen auf nicht von uns ausgeteilten Blättern nicht zu berücksichtigen. Die **Klausureinsicht** findet am 14.10.2024 von 14 bis 17 Uhr in Raum 010 im Informatik-Gebäude statt.

Aufgabe 1 (Haskell: Newton-Iteration) [10 Punkte]

Die Newton-Iteration zur Nullstellenbestimmung einer differenzierbaren Funktion verwendet die Folge $(x_n)_{n\in\mathbb{N}}$ mit vorgegebenem Startwert x_0 und Iterationsvorschrift $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.

Sie sollen in dieser Aufgabe die beiden Haskell-Funktionen `folge` und `abbruch` implementieren, so dass die Definition

```
newton :: (Double -> Double) ->
          (Double -> Double) ->
          Double ->
          (Double -> Double -> Bool) ->
          Double
newton f f' x0 p = abbruch p (folge f f' x0)
```

zur Berechnung von Nullstellen vom ersten Parameter `f` genutzt werden kann.

(a) Implementieren Sie die Funktion [4 Punkte]

```
abbruch :: (Double -> Double -> Bool) -> [Double] -> Double
```

`abbruch p xn` gibt für die unendliche Liste `xs` das erste Element `xs !! (n+1)` zurück[1], für welches das Abbruchkriterium `p (xs !! n) (xs !! (n+1))` wahr ist.

(b) Implementieren Sie die Funktion [6 Punkte]

```
folge :: (Double -> Double) -> (Double -> Double) -> Double -> [Double]
```

`folge f f' x0` berechnet die Newton-Iterations-Folge $(x_n)_{n\in\mathbb{N}}$ von oben als unendliche Liste.

Beispiel:

```
let f  = \x -> x*x-2
    f' = \x -> 2*x
in take 4 (folge f f' 1.0) == [1.0,1.5,1.4166666666666667,1.414215...]
```

Hinweis: Verwenden Sie `iterate :: (a -> a) -> a -> [a]` (siehe auch Vorlesungsfolie 12.09f) mit geeigneter Hilfsfunktion.

[1] falls ein solches existiert

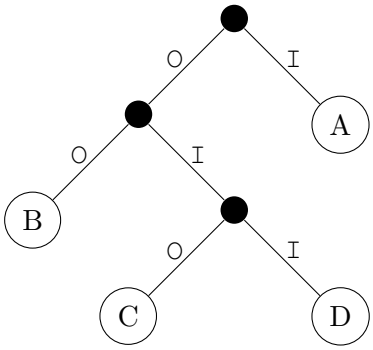
Aufgabe 2 (Haskell: Huffman-Codierung) [24 Punkte]

Huffman-Codes sind ein Verfahren zur Datenkompression. Sie encodieren einen Eingabe-Zeichenstrom in einen Strom von Bits, z.B. `"aba"` als `1001`. Die Codierung erfolgt Zeichen für Zeichen (z.B. `'b'` durch `00`) mithilfe eines binären Huffman-Baums, der folgendermaßen definiert ist:

```
data HTree = Leaf Char | Inner HTree HTree
data Bit = O | I
```

Die Blätter des Baumes enthalten die möglichen Eingabezeichen. Wir markieren linke Kanten innerer Knoten mit Bit `O` und rechte Kanten mit Bit `I`. Dann ergibt sich die Codierung eines Zeichens aus dem Pfad `p :: [Bit]` von der Wurzel zum zugehörigen Blatt. **Beispiel:**

Der nebenstehende Huffman-Baum `ht` repräsentiert folgende Codierung:




```
'A'  ↔  [I]
'B'  ↔  [O, O]
'C'  ↔  [O, I, O]
'D'  ↔  [O, I, I]
```

Als Haskell-Datum sieht der Baum so aus:

```
ht = Inner (Inner (Leaf 'B')
                  (Inner (Leaf 'C') (Leaf 'D')))
           (Leaf 'A')
```

(a) Implementieren Sie eine Funktion [8 Punkte]

```
decode :: HTree -> [Bit] -> [Char]
```

die eine potenziell unendliche Liste von Bits einliest, entsprechend des gegebenen Huffman-Baums decodiert und die Liste der decodierten Zeichen zurückgibt. Wenn die Eingabe mitten in einem Zeichen endet, ignorieren Sie die überzähligen Bits.
Beispiel: `decode ht [I, O, O, I] == "aba"`.

In den folgenden Teilaufgaben dürfen Sie den Fall ignorieren, dass ein Eingabezeichen nicht oder mehrfach im Huffman-Baum vorkommt.

(b) Nehmen Sie an, dass Sie eine Funktion [3 Punkte]

```
encodeOne :: HTree -> Char -> [Bit]
```

gegeben haben, die ein einzelnes Zeichen entsprechend des gegebenen Huffman-Baums codiert. Implementieren Sie mit ihrer Hilfe eine Funktion

```
encode :: HTree -> [Char] -> [Bit]
```

die eine potenziell unendliche Liste von Zeichen im Huffman-Code codiert.

(c) Implementieren Sie nun die Funktion `encodeOne` wie oben beschrieben. [13 Punkte]

Hinweise:

- Zwei mögliche Ansätze um die Funktion `encodeOne` zu implementieren sind:
  - Konvertieren Sie den Huffman-Baum in eine Liste von Paaren `(Char, [Bit])` und finden Sie in dieser Liste das zu encodierende Zeichen.
  - Verwenden Sie den `Maybe`-Typen, um anzugeben, ob in einem Unterbaum das gesuchte Zeichen gefunden wurde, und benutzen Sie den `first`-Kombinator um die Ergebnisse zusammenzuführen. Dieser ist wie folgt definiert:

    ```
    first Nothing  y = y
    first (Just x) _ = Just x
    ```

- Sie können einen Akkumulator benutzen um den Baumpfad `p :: [Bit]` für das Codewort aufzubauen.

Aufgabe 3 (Prolog: Eimer) [18 Punkte]

Gegeben seien drei Eimer mit den Fassungsvermögen 8, 5 bzw. 3 Liter. Anfangs sei der Eimer mit 8 Litern voll, die anderen leer. Wie kann man durch Um- und Ausschütten 4 Liter in einem Eimer erhalten?

Der Füllzustand der Eimer wird durch ein Tripel `(X,Y,Z)` repräsentiert. Zum Beispiel bedeutet `(6,1,0)`, dass im 8-Liter-Eimer 6 Liter sind, der 5-Liter-Eimer einen Liter enthält und der 3-Liter-Eimer leer ist.

(a) Definieren Sie einstellige Prolog-Prädikate `start` und `final`, [3 Punkte]
so dass `start(F)` genau für den Füllstand zu Anfang erfüllt ist und `final(F)` genau dann erfüllt ist, wenn ein Endzustand mit 4 Litern in einem Eimer erreicht ist.

(b) Definieren Sie ein dreistelliges Prolog-Prädikat `move(F1,M,F2)` [7 Punkte]
welches angibt, dass sich aus dem Füllzustand `F1` der Füllzustand `F2` durch einmaliges Um- bzw. Ausschütten eines Eimers ergeben kann. `M` ist ein String, der die Umfüllaktion beschreibt. Geben Sie dazu Regeln an zum

- Ausschütten von nicht-leeren Eimern,
- Umschütten von Eimern in andere Eimer. Dabei wird genauso viel Wasser umgeschüttet wie im Zieleimer noch Platz ist – restliches Wasser verbleibt im Quelleimer.

Es reicht, wenn Sie die Regeln zum Ausschütten vom 3-Liter-Eimer und zum Umschütten vom 8-Liter-Eimer in den 3-Liter-Eimer angeben.

Beispiel:

```
? move((7,0,1),M,FL).
M = 'E3 ausschuetten',
FL = (7, 0, 0) ;

M = 'E8 -> E5',
FL = (2, 5, 1) ;

M = 'E8 -> E3',
FL = (5, 0, 3) ;
...
```

(c) Definieren Sie ein Prolog-Prädikat `solve`, [8 Punkte]
das eine Antwort auf die Frage „Wie erhält man 4 Liter in einem Eimer?“ berechnet. `solve(F0, FL, Ms, Fs)` ist dann erfüllt, wenn sich der Füllzustand `F0` mittels der Füllzustandsübergänge von `move` nach `FL` überführen lässt, ohne dass einer der Zwischenzustände in der Liste `Fs` besucht wird. Die Liste `Ms` sammelt dabei die Beschreibungen der einzelnen `moves`. Die Liste `Fs` der verbotenen Zustände enthält die bereits besuchten Zustände, um Endlosschleifen zu vermeiden.

Mit diesem Prädikat lässt sich dann das Anfangsproblem wie folgt lösen:

```
solution(Ms) :- start(F0), final(FL), solve(F0, FL, Ms, [F0]).
```

Hinweis: Verwenden Sie die Prädikate `member` und `not` aus der Vorlesung.

Aufgabe 4 (Reduktionsgraphen im λ-Kalkül) [10 Punkte]

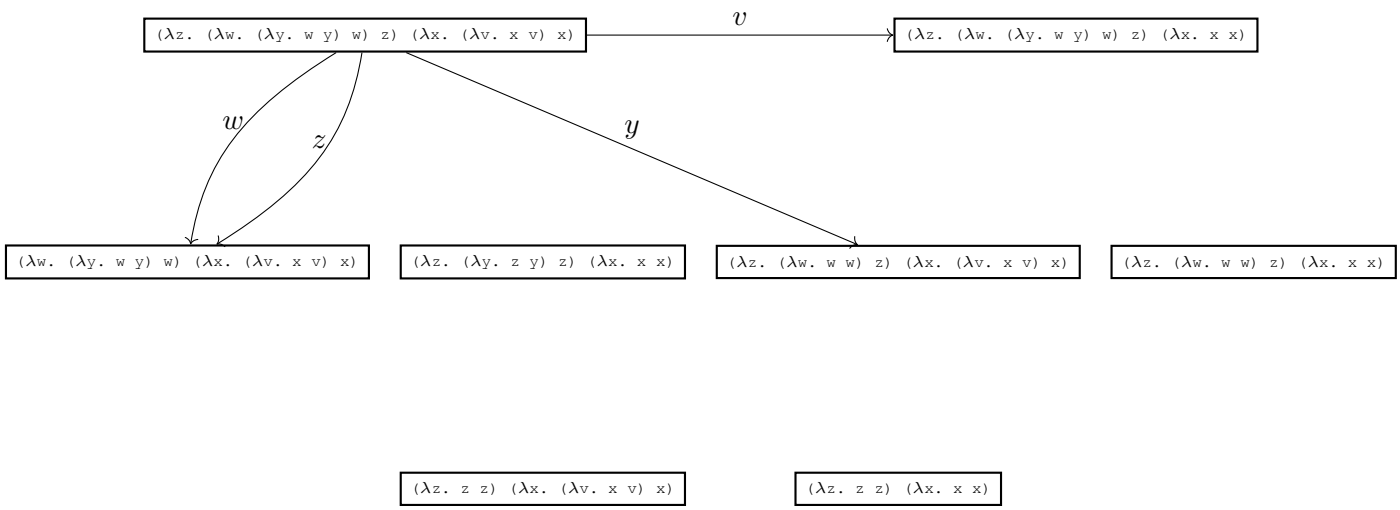
Für einen λ-Term t sei der *Reduktionsgraph* (V_t, E_t) von t wie folgt definiert:

- Die Knotenmenge V_t enthält t und alle λ-Terme, welche sich durch wiederholte β-Reduktion von t erreichen lassen. Hierbei werden *alle* möglichen Reduktionen der jeweiligen Terme betrachtet. α-äquivalente Terme werden *vom selben Knoten* repräsentiert.
- Die Kantenmenge E_t enthält alle zulässigen β-Reduktionsschritte. Es werden nicht mehrere Schritte zusammengefasst. Unterschiedliche Redexe führen zu unterschiedlichen Kanten.

Zeichnen Sie umseitig die Kanten des Reduktionsgraphen für den folgenden Term ein:

$$t = (\lambda \mathtt{z.}\ (\lambda \mathtt{w.}\ (\lambda \mathtt{y.}\ \mathtt{w\ y)\ w)\ z)\ (\lambda x.\ (\lambda v.\ x\ v)\ x)}$$

Beschriften sie hierbei jede Kante mit der λ-Variable, welche durch die entsprechende β-Reduktion ersetzt wird.



Aufgabe 5 (Typinferenz) [15 Punkte]

(a) Gegeben ist die folgende Menge von Typgleichungen: [4 Punkte]

$$\begin{aligned} C = \{\, & \alpha_1 = \alpha_2 \to \alpha_3, \\ & \alpha_3 = \alpha_4 \to \alpha_5, \\ & \alpha_5 = \alpha_6 \to \alpha_7, \\ & \alpha_8 = \alpha_9 \to \alpha_7, \\ & \alpha_4 = \alpha_8, \\ & \alpha_2 = \alpha_9 \,\} \end{aligned}$$

Geben Sie einen allgemeinsten Unifikator σ von C an. Rechnen Sie ihn vollständig aus, d. h. geben Sie ihn in der Form

$$[\ldots, \alpha_i \Rsh \ldots, \ldots]$$

an (Zuordnungen der Form $\alpha_i \Rsh \alpha_i$ dürfen Sie weglassen).

(b) Geben Sie einen allgemeinsten Typen τ_{just} von [5 Punkte]

$$just = \lambda \mathtt{x}.\ \lambda \mathtt{j}.\ \lambda \mathtt{n}.\ \mathtt{j}\ \mathtt{x}$$

an.

(c) Zeigen Sie, dass der Term [6 Punkte]

$$\textbf{let}\ \mathtt{just} = \lambda \mathtt{x}.\ \lambda \mathtt{j}.\ \lambda \mathtt{n}.\ \mathtt{j}\ \mathtt{x}\ \textbf{in}\ \mathtt{just}\ \mathtt{0}$$

typisierbar ist. Vervollständigen Sie dafür die grauen Boxen in folgendem Herleitungsbaum und geben Sie Γ_{let} an:

$$\textit{Let}\ \dfrac{\textit{Abs}\ \dfrac{\ldots}{\vdash \lambda \mathtt{x}.\ \lambda \mathtt{j}.\ \lambda \mathtt{n}.\ \mathtt{j}\ \mathtt{x} : \tau_{just}} \qquad \textit{App}\ \dfrac{\textit{Var}\ \dfrac{\boxed{} = \boxed{} \quad \boxed{} \succeq \boxed{}}{\Gamma_{let} \vdash \mathtt{just} : \boxed{}} \qquad \dfrac{\ldots}{\Gamma_{let} \vdash \mathtt{0} : \mathsf{int}}}{\Gamma_{let} \vdash \mathtt{just}\ \mathtt{0} : \boxed{}}}{\vdash \textbf{let}\ \mathtt{just} = \lambda \mathtt{x}.\ \lambda \mathtt{j}.\ \lambda \mathtt{n}.\ \mathtt{j}\ \mathtt{x}\ \textbf{in}\ \mathtt{just}\ \mathtt{0} : \boxed{}}$$

$\Gamma_{let} = \boxed{}$

Aufgabe 6 (MPI) [10 Punkte]

Gegeben sei das untenstehende Programm.
Hinweis: Die Methode `stringtolong(const char *str)` konvertiert den String in `str` zu einem `long`. Sie können davon ausgehen, dass die Methode nur mit gültigen Parametern aufgerufen wird. Weiterhin können Sie davon ausgehen, dass `malloc` nicht `NULL` zurückgibt.

```
1  int main(int argc, char *argv[]) {
2      int N = stringtolong(argv[1]);
3      int rank, size;
4
5      MPI_Init(&argc, &argv);
6      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
7      MPI_Comm_size(MPI_COMM_WORLD, &size);
8
9      long *local_results = (long *)malloc(sizeof(long) * size);
10     long *numbers = (long *)malloc(sizeof(long) * N);
11     long global_result;
12
13     if (rank == 0) {
14         for (long i = 0; i < N; i++) {
15             numbers[i] = i + 1;
16         }
17     }
18
19     long *local_numbers = (long *)malloc(sizeof(long) * (N / size));
20     MPI_Scatter(numbers, N / size, MPI_LONG, local_numbers, N / size,
          MPI_LONG, 0, MPI_COMM_WORLD);
21
22     long local_result = 1;
23     for (int i = 0; i < N / size; i++) {
24         local_result *= local_numbers[i];
25     }
26
27     *(local_results + rank) = local_result;
28     MPI_Gather(local_results + rank, 1, MPI_LONG, local_results, 1,
          MPI_LONG, 0, MPI_COMM_WORLD);
29
30     if (rank == 0) {
31         for (long i = 0; i < size; i++) {
32             global_result *= local_results[i];
33         }
34         printf("Result for %d: %ld\n", N, global_result);
35     }
36
37     free(local_results); free(numbers); free(local_numbers);
38     MPI_Finalize();
39     return 0;
40 }
```

(a) Füllen Sie in den nachfolgenden Tabellen aus, wie die jeweils angegebenen Variablen nach Ausführung der in den Tabellen gegebenen Quelltext-Zeile belegt sind für eine Ausführung des Programms mit 4 Prozessen und 8 als übergebenem Parameter. Markieren Sie ggf. unbekannte Werte von Speicherstellen mit einem –: Falls der Wert aller Speicherstellen einer Variable unbekannt ist, genügt ein –. Machen Sie ansonsten kenntlich, an welchen Stellen Werte unbekannt sind. *Hinweis:* Es genügt die Berechnung aufzuschreiben, Sie müssen die Ergebnisse nicht ausrechnen. [5 Punkte]

(a) **Variable: `numbers`**

Prozess Nr.	**Wert nach Zeile 17**
0	
1	
2	
3	

(b) **Variable: `local_numbers`**

Prozess Nr.	**`Wert nach Zeile 20`**
0	
1	
2	
3	

(c) **Variable: `local_result`**

Prozess Nr.	**`Wert nach Zeile 25`**
0	
1	
2	
3	

(d) **Variable: `local_results`**

Prozess Nr.	**`Wert nach Zeile 28`**
0	
1	
2	
3	

(e) **Variable: `global_result`**

Prozess Nr.	**`Wert nach Zeile 35`**
0	
1	
2	
3	

(b) Geben Sie eine mathematische Formel für die vom Programm für den Eingabeparameter `N` realisierte Berechnung an. [1 Punkt]

(c) Durch welche MPI-Operation lässt sich die Programmlogik von Zeile 27 bis Zeile 35 ersetzen, sodass die Funktionalität des Programms erhalten bleibt? Den Aufruf von `printf` in Zeile 34 müssen Sie dabei nicht berücksichtigen. Geben Sie die MPI-Operation sowie deren Parametrisierung an. [2 Punkte]

(d) Ist es sinnvoll, das `numbers` array auf Prozess 0 zu initialisieren und dann auf alle anderen Prozesse zu verteilen? Begründen Sie. Wie können Sie die Parallelisierung im gegebenen Programm zwischen Zeile 11 und 22 gegebenenfalls weiter verbessern, sodass weniger Synchronisation und Kommunikation zwischen den Prozessen benötigt wird? Eine Beschreibung Ihrer Verbesserungen ist ausreichend, Sie müssen keinen Code oder Formeln schreiben. [2 Punkte]

Aufgabe 7 (Java) [15 Punkte]

In dieser Aufgabe wird die parallele Abarbeitung von Aufgaben in einem Online-Shop für Tickets (z.B. Konzerttickets) betrachtet. Dabei ist darauf zu achten, dass nicht mehr als die verfügbare Menge an Tickets verkauft wird und ein Sitzplatz nicht doppelt belegt wird. Tickets können dabei mit oder ohne Sitzplatzreservierung gebucht werden.

(a) Im nachfolgenden Code werden verschachtelte kritische Abschnitte mit `synchronized`-Blöcken geschützt. Nun sollen die kritischen Abschnitte stattdessen mit Locks geschützt werden. Die Abschnitte, in denen die Locks gehalten werden, sollen dabei analog zu den Abschnitten der synchronisierten Ressourcen sein. Insbesondere darf ein Lock nicht länger gehalten werden, als die entsprechende Ressource durch den zugehörigen `synchronized`-Block. [3 Punkte]

Geben Sie die notwendigen Aufrufe der Methoden `Lock::lock`, bzw. `Lock::unlock` an, indem Sie sie in die dafür vorgesehenen Felder eintragen. Verwenden Sie `ticketsLock` und `seatsLock` als Locks für den ersten, bzw. zweiten kritischen Abschnitt. Sie dürfen mehr als einen Aufruf pro Feld angeben und müssen nicht alle Felder für eine korrekte Lösung ausfüllen.

```
1  void reserveWithRandomSeats(int amount) {
2      synchronized (tickets) {
3          if (!tickets.areAvailable(amount))
4              return;
5          Reservation reservation = tickets.reserve(amount);
6          synchronized (seats) {
7              try {
8                  seats.choose(reservation);
9              } catch (NoSeatsException ex) {
10                 reservation.cancel();
11                 return;
12             }
13         }
14         reservation.confirm();
15     }
16 }
```

(b) Zur Synchronisierung des Zugriffs auf Ressourcen zur Buchung von Sitzplätzen und zusätzlichen Leistungen werden die Locks `seatsLock` und `extrasLock` eingesetzt. In zwei Methoden, die in verschiedenen Threads ausgeführt werden, werden die Locks allerdings unterschiedlich verwendet, wie in dem nachfolgenden Code zu sehen ist. **Beschreiben Sie, warum es dabei zu einem Deadlock kommen kann und geben Sie 2 Vorschläge, wie dies verhindert werden könnte**. Nehmen Sie in jedem Vorschlag Bezug auf eine der vier Coffman-Bedingungen. Sie müssen in Ihrer Lösung keinen Programmcode schreiben. [3 Punkte]

	Thread A	**Thread B**
1	`seatsLock.lock();`	`extrasLock.lock();`
2	`seats.choose(reservation);`	`seatsLock.lock();`
3	`if (extras.isPresent()) {`	`seats.choose(reservation);`
4	`  extrasLock.lock();`	`seatsLock.unlock();`
5	`  extras.addTo(reservation);`	`extras.addTo(reservation);`
6	`}`	

(a) *Ihre Lösung:*

```
if (!tickets.areAvailable(amount)) {
    return;
}
Reservation reservation = tickets.reserve(amount);
try {
    seats.choose(reservation);
} catch (NoSeatsException ex) {
    reservation.cancel();
    return;
}
reservation.confirm();
```

(b) *Ihre Lösung:*

(c) In dem nachfolgenden Code wird die Erzeugung der Buchungsbestätigungen beschrieben. Dafür werden die beiden Futures `futureTicket` und `futureSeats` verwendet, die in unabhängigen, jeweils neu erzeugten Threads ausgeführt werden. Sie können davon ausgehen, dass die Abarbeitung des Futures `futureTicket` beendet ist, bevor mit der Bearbeitung des Futures `futureSeats` begonnen wird. [4 Punkte]

Geben Sie alle happens-before-Beziehungen an, die das Statement in Zeile 6 zu den vorherigen und nachfolgenden Statements im Code hat, indem Sie Pfeile (⟶) zwischen den Punkten (○/●) hinter den Zeilen einzeichnen. **Geben Sie außerdem an, wenn zwischen dem Statement in Zeile 6 und einem anderen Statement keine happens-before-Beziehung besteht**, indem Sie eine Linie mit einem Kreuz (–✕–) zwischen den entsprechenden Punkten einzeichnen. Ein Punkt am Ende eines mehrzeiligen Statements gilt für das gesamte Statement. Wenn ein Statement a eine happens-before-Beziehung zu Statement b hat (b sieht das Ergebnis von a), zeichnen Sie einen Pfeil von a zu b.

Hinweis: Überlegen Sie sich, wann für die Ausführung eines Futures ein Thread gestartet, bzw. auf einen Thread gewartet wird.

```
1  prepareWebpage();                                  ●
2  Future<Document> futureTicket =
3      executor.submit(() -> {
4          PersonalData data = getPersonalData();     ●
5          synchronized (reservation) {
6              return generateTicketConfirmation(
                   data, reservation);                ○
7          } });
8  Future<Document> futureSeats =
9      executor.submit(() -> {
10         PersonalData data = getPersonalData();     ●
11         synchronized (reservation) {
12             return generateSeatsConfirmation(
                   data, reservation);                ●
13         } });
14 updateWebpage();                                   ●
15 Document ticket = futureTicket.get();
16 startDownload(ticket);                             ●
```

(d) Für einige Methoden, die im Buchungsprozess benutzt werden, existieren bereits JML-Verträge, die das Verhalten der Methoden spezifizieren. Diese sollen nun in der Methode `reserveWithRandomSeats` benutzt werden, um Tickets, sowie Sitzplätze zu reservieren und die Reservierung zu bestätigen. [5 Punkte]

Implementieren Sie die Methode `reserveWithRandomSeats` unter Verwendung der angegebenen Methoden. Berücksichtigen Sie dabei die JML-Verträge der aufgerufenen Methoden. Verwenden Sie alle angegebenen Methoden und implementieren Sie nur die in der Aufgabe geforderte Funktionalität.

Geben Sie außerdem eine möglichst schwache Vor- und eine möglichst starke Nachbedingung für die Methode `reserveWithRandomSeats` an, die sich aus den Vor- und Nachbedingungen der aufgerufenen Methoden herleiten lässt. Geben Sie keine trivialen Bedingungen wie `requires true` an. Falls die Methode nicht weiter ausgeführt werden kann, beenden Sie sie sofort und geben Sie den Wert `null` zurück.

```
1  /*@ requires reservation.hasTickets();
2   *@ ensures reservation.hasSeats() <==> \result;
3    @*/
4  boolean chooseSeats(Reservation reservation) { /* ... */ }
5
6  /*@ requires amount > 0;
7   *@ ensures \result.hasTickets();
8    @*/
9  Reservation reserveTickets(int amount) { /* ... */ }
10
11 /*@ requires reservation.hasTickets();
12  *@ requires reservation.hasSeats();
13   @*/
14 void confirm(Reservation reservation) { /* ... */ }
15
16 /*@ requires
17  *@ ensures
18   @*/
19 Reservation reserveWithRandomSeats(int amount) {
20 }
```

Aufgabe 8 (Syntaxanalyse: Reguläre Ausdrücke) [18 Punkte]

Eine Grammatik für einfache reguläre Ausdrücke ist (Terminale sind unterstrichen):

$$R \;\rightarrow\; \underline{\textbf{char}} \;|\; \underline{\varepsilon} \;|\; \underline{(}\; R\; \underline{\cup}\; R\; \underline{)} \;|\; \underline{(}\; R\; \underline{\cdot}\; R\; \underline{)} \;|\; \underline{(}\; R\; \underline{)}\; \underline{*}$$

Zusammengesetzte Ausdrücke müssen also geklammert werden. Nach Linksfaktorisierung erhält man:

$$\begin{aligned} R &\rightarrow \underline{\textbf{char}} \;|\; \underline{\varepsilon} \;|\; \underline{(}\; R\, R' \\ R' &\rightarrow \underline{\cup}\; R\; \underline{)} \;|\; \underline{\cdot}\; R\; \underline{)} \;|\; \underline{)}\; \underline{*} \end{aligned}$$

(a) Ist die linksfaktorisierte Grammatik SLL(1)? Begründen Sie kurz anhand der Indizmengen. [2 Punkte]

Der AST-Datentyp für reguläre Ausdrücke sei vorgegeben als:

```
abstract class RegExp {}
class LiteralChar extends Regexp { Char lit; }
class EmptyString extends Regexp {}
class Union extends Regexp { RegExp l; RegExp r; }
class Concatenation extends Regexp { RegExp l; RegExp r; }
class KleeneClosure extends Regexp { RegExp e; }
```

Sie dürfen annehmen, dass jede Klasse einen Konstruktor hat, der die Felder in ihrer Deklarationsreihenfolge zuweist. Zum Beispiel hätte die Klasse `class C { String a; int b; }` implizit einen Konstruktor `C (String a, int b) { this.a = a; this.b = b; }`.

(b) Geben Sie für jedes Nichtterminal der Grammatik einen rekursiven Abstiegsparser an, der einen solchen AST aufbaut. Brechen Sie bei Parserfehlern mit der globalen `error()`-Funktion ab, sobald das aktuelle Token in keiner Indizmenge liegt. [16 Punkte]

Hinweis: Sie können davon ausgehen, dass die Lexer-Schnittstelle wie folgt aussieht:

- Die Token sind Werte eines `enum` namens **`Token`** und heißen **`LP`**, **`RP`**, **`UNION`**, **`DOT`**, **`STAR`**, **`EPSILON`**, **`CHAR`** und **`EOF`** für $\underline{(}$, $\underline{)}$, $\underline{\cup}$, $\underline{\cdot}$, $\underline{*}$, $\underline{\varepsilon}$, sonstige Zeichen $\underline{\textbf{char}}$, sowie das Ende der Eingabe. Token können bspw. mit `==` verglichen werden.
- `lexer.current` gibt nichtkonsumierenden Zugriff auf das aktuelle Token.
- `lexer.lex()` konsumiert das aktuelle Token.
- `lexer.char` enthält das zu einem **`CHAR`** gehörende Zeichen als `Char`.
- Die folgende Funktion ist verfügbar:

```
Char expect(Token tok) {
    if (lexer.current != tok) error();
    Char c = lexer.char;
    lexer.lex();
    return c;
}
```